IE535 Linear Optimization
Programming Project

Fall 2026 | Purdue University School of Industrial Engineering

Deadline: Thursday, Dec. 17, 2026, 11:59 PM EST

Submission: through **Brightspace!** NOT Gradescope

Choose one of two tracks:

(i) the naive simplex method, or
(ii) the centered interior point method.

Develop your own LP examples, use an AI coding tool to help implement the selected algorithm, and verify your results mathematically and with a commercial solver. These requirements apply to both on-campus and online students. There is no extra credit.

1. Common requirements

➢ Problem selection and size

Construct the examples required by your chosen track. **Each example must contain at least eight original decision variables and five substantive constraints (that is, other than nonnegativity constraints of the variables).** You may develop an application from your research or professional work, or construct a mathematical example designed to demonstrate a specified LP behavior.

- Count variables before adding slack, surplus, or artificial variables or splitting unrestricted variables. Nonnegativity declarations do not count toward the constraint minimum.
- Related examples obtained by modifying one model are acceptable. Explain each modification and why it produces the required behavior. Identify the source of any adapted example or data (if adapted from others' examples). Treat all variables as continuous.

For every example, provide the objective, all constraints, variable definitions, input data, and expected outcome.

➢ Standard form and implementation

Use the following minimization convention for the algorithm:

$$min\ c^T x; \quad Ax = b, x \geq 0.$$

Here *A* has *m* rows and *n* columns, *b* is the right-hand-side vector, *c* is the objective coefficient vector, and *x* is the vector of standard-form variables. Document the correspondence between original and standard-form variables. Manual conversion is permitted.

- ➢ **Programming language – Only use Python, Julia, MATLAB, or R.** Use at least one AI coding assistant or agent to develop the implementation and retain all project prompts.
- ➢ **Commercial solver verification** -- Solve every submitted LP with any commercial solver, such as Gurobi, CPLEX, MOSEK, or MATLAB Optimization Toolbox's linprog. Submit the separate solver script, solver name and version, invocation, settings, returned status, and numerical results.

2. AI documentation and submission

Use at least one AI coding assistant or agent to develop your implementation. You may use AI for model construction, mathematical reasoning, code generation, testing, and debugging. You remain responsible for every model, algorithmic decision, numerical result, and claim in your submission. You must understand your implementation and be able to explain its key calculations.

Submit a searchable PDF record of all prompts you supplied for this project, including follow-up prompts and instructions to coding agents. Include the corresponding responses or available session exports so that the development process can be followed. Begin retaining these records when you start; do not reconstruct a hypothetical prompt history afterward.

Include a brief explanation of important AI-generated suggestions you accepted, corrected, or rejected and how you verified them. AI-generated comments and explanations must be checked for mathematical accuracy.

3. Files to submit through Brightspace

- Project report PDF: student name and ID; chosen Project Option; all models and data; algorithm explanation; verification calculations; iteration results; commercial-solver comparisons.
- Runnable source files: the algorithm, a driver that runs all required examples or starting points, the separate commercial-solver script, and every required input data file. Submit editable source files as well as PDFs. Do not submit only a notebook, executable, screenshots, or links to externally hosted code.
- AI record PDF: all project prompts and corresponding responses or session exports, organized by tool and conversation.

****

Project Option 1 (OP1) -- Track A Naive simplex implementation requirements

OP1.(a) You must use the Two-Phase approach

Implement one reusable two-phase simplex solver and use it for all four examples specified in the OP1 requirements below. Your code must handle initialization, infeasibility, dependent equalities, degeneracy, finite optimality, and unboundedness.

Basis calculations and pivoting

- At every iteration, form the basis matrix B from an ordered list of basic columns and explicitly calculate its inverse from scratch. Built-in matrix inversion is permitted. If you want to implement tableau pivoting or other efficient numerical methods to avoid directly inversing the matrix B, please feel free to do so.
- Report the basic-variable values and reduced costs from at each iteration.
- Implement the Bland's rule: choose the eligible entering variable with smallest original index. Among variables tied in the minimum ratio test, choose the leaving variable with smallest original index, rather than the smallest row position. Retain the original variable indices after each pivot.

Phase I and dependent equalities

- Use the two-phase method to obtain an initial feasible basis or establish infeasibility. Normalize negative right-hand sides as needed and document your artificial-variable construction.
- If the minimum Phase I artificial-variable sum is positive beyond the stated tolerance, report infeasibility. Otherwise, remove artificial variables before Phase II.
- Handle zero-valued artificial variables that remain basic at the end of Phase I. Pivot them out when possible; remove redundant equalities when necessary. Detect inconsistent equalities rather than silently

deleting them. Explain why any row removed is redundant in the augmented system, including its right-hand side.

Termination and iteration output

Distinguish optimal, infeasible, and unbounded outcomes. Report iteration-limit or numerical failure separately; neither establishes infeasibility. State tolerances for feasibility, reduced costs, pivot eligibility, ties, and rank decisions.

For every Phase I and Phase II iteration, save the phase, iteration number, ordered basis indices, primal vector, objective value, reduced costs, entering and leaving indices, and step length. Include the final termination reason. At termination, output the applicable checks or certificate described in Section 3.

OP1.(b) Required simplex examples and verification

Submit **four** examples. Every example must meet the size and modeling requirements in **Section 1**. (Rank deficiency is a property of the standard-form equality matrix; it is separate from the LP's solution status.)

Example 1. A finite optimal solution

Construct a feasible LP with a finite optimal value. In addition to printing out the basis and reduced costs at each iteration, also report the optimal solution, objective value, final basis, and the final reduced costs. Verify feasibility and nonnegative reduced costs within your stated tolerances. Explain why the final point is optimal.

Example 2. An unbounded objective

Construct a feasible LP whose minimization objective is unbounded below. Output a feasible point x^o and an improving recession direction d. For the nonempty feasible set P, state the recession cone and print its defining constraints using your actual data:

$$P = \{x : Ax = b, x \geq 0\}, \quad rec(P) = \{d : Ad = 0, d \geq 0\}.$$

Verify the following conditions numerically and explain their mathematical meaning:

$$Ax^0 = b, \quad x^0 \geq 0, \quad Ad = 0, \quad d \geq 0, \quad c^T d < 0.$$

Explain why $x^o + td$ remains feasible for every $t \geq 0$ and why its objective tends to negative infinity.

Example 3. An infeasible LP

Construct an LP with no feasible solution. Report the Phase I optimal objective value and artificial-variable values, and explain why feasibility is impossible.

Example 4. Dependent equality constraints

Construct a feasible LP with a finite optimum whose standard-form equality matrix does not have full row rank. Show an explicit dependence among its rows and the corresponding right-hand-side entries. Identify the redundant equalities removed by your program and display the reduced system. Solve it and compare its optimal value with the commercial solver applied to the original example (that is, you send the rand deficient example directly to a commercial LP solver).

Project Option 2 (OP2) -- Centered interior point implementation

Construct one LP with a finite optimal solution and implement a centered, infeasible-start primal-dual path-following interior point method. Meet the size requirements in Section 1. For this track, choose a full-row-rank standard-form matrix and an LP with a feasible point.

OP2.(a) Algorithm and initialization

Use the centered path-following algorithm, rather than an affine-scaling method. Implement the Newton system, centering rule, updates, positivity-preserving step selection, and stopping criteria yourself. Built-in linear system solvers and matrix factorizations are permitted.

For the minimization primal in Section 1, use the dual maximize $b^T y$ subject to $A^T y + s = c$ and $s \geq 0$. Here y is the vector of equality multipliers and s is the vector of dual slacks. The initial values must satisfy:

$$x^0 > 0, \quad s^0 > 0, \quad y^0 \; arbitrary.$$

The starting point must NOT satisfy the primal or dual equality constraints. The routine must accept user-supplied starting values and must not require a feasibility solve before starting. “Arbitrary” refers to initialization within these positivity requirements; zero or negative primal variables or dual slacks are not admissible interior starting values.

Let *X* and *S* denote diagonal matrices with diagonals *x* and *s*, let e be the all-ones vector of length *n*, and let μ be the average complementarity. Document the centering parameter σ and its update rule. The centered complementarity target is:

$$XSe = \sigma\mu e, \quad \mu = x^T s / n, \quad 0 < \sigma < 1.$$

OP2.(b) TWO starting points and convergence evidence

Solve the same LP from TWO substantially different positive starting points, each violating at least one primal or dual equality.

- For every iteration, print out x, y, s, primal and dual objective values, primal and dual feasibility residuals, average complementarity, centering parameter, and step lengths.
- Terminate successfully only when primal feasibility, dual feasibility, and complementarity satisfy your documented tolerances. A small complementarity value alone does not establish optimality. Distinguish numerical failure or an iteration limit from success.

This track does not require detection of infeasible or unbounded LPs or automatic redundancy removal.